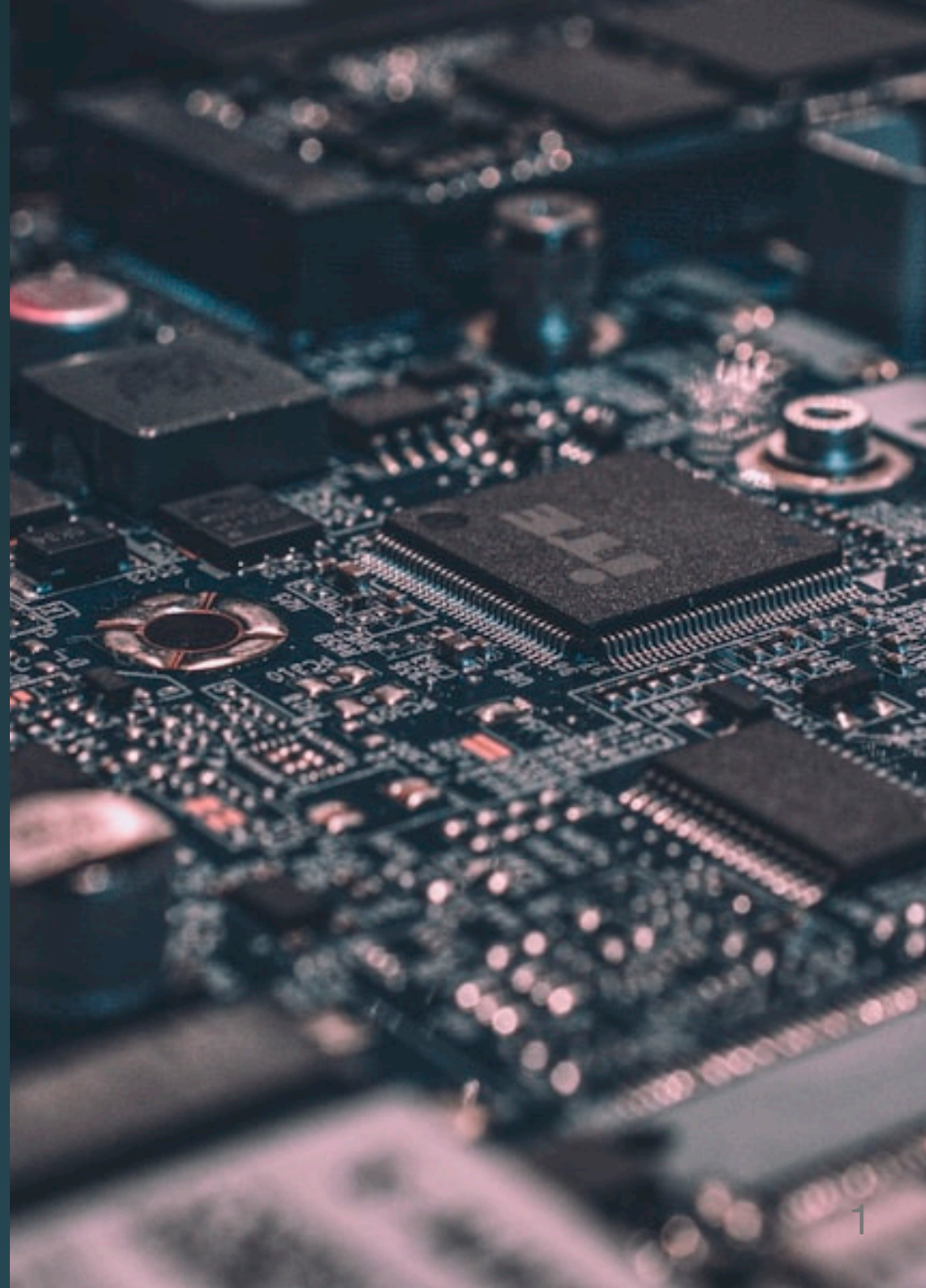


Scrape Pipeline

Lesson 09 · ISBA 4715



Agenda

	Part	What
1	Scraping Concepts	What it is, etiquette, the tools we'll use (<i>slides</i>)
2	Python Pipeline	Tavily + Firecrawl, saving to <code>knowledge/raw/</code> (<i>live code</i>)
3	MCP Upgrade	Claude Code as the pipeline (<i>demo</i>)
4	Your Project	Scrape at least one source into your repo

How did Google build its first index?



Before anyone wrote an API for the web...



How did Google build its first index?



Before anyone wrote an API for the web...

Web scraping — reading pages programmatically.



Web Scrapping

Gathering data from websites **without an API** and **without a human clicking around in a browser**

Why Scrape?



No API exists for this data



Constantly-changing list (press releases, bios, news)



The API is paywalled



You want to automate something a human would do

Etiquette and Caveats

Rule	Why
Check <code>robots.txt</code>	Sites declare what bots can crawl
Rate limit — <code>time.sleep()</code>	Don't hammer the server
Respect Terms of Service	Legal + ethical obligation
Expect layout changes	HTML is not a stable contract
Be mindful of PII	Don't scrape what people did not consent to share



**Where does scraped data go in
THIS project?**

Where does scraped data go?

MP03 (API)

request → CSV → SQL warehouse

Where does scraped data go?



Where does scraped data go?



These feed your **knowledge base** — the unstructured side of your portfolio project.

The Two-Step Pattern



Discover: find URLs worth scraping → **Tavily**

Extract: turn each URL into clean markdown → **Firecrawl**

Tavily — AI-Native Search API



You send: a **query**

You get back: **JSON** with ranked URLs, titles, content previews

```
{"results": [  
  {"url": "...", "title": "...", "content": "...", "score": 0.87},  
  ...  
]}
```

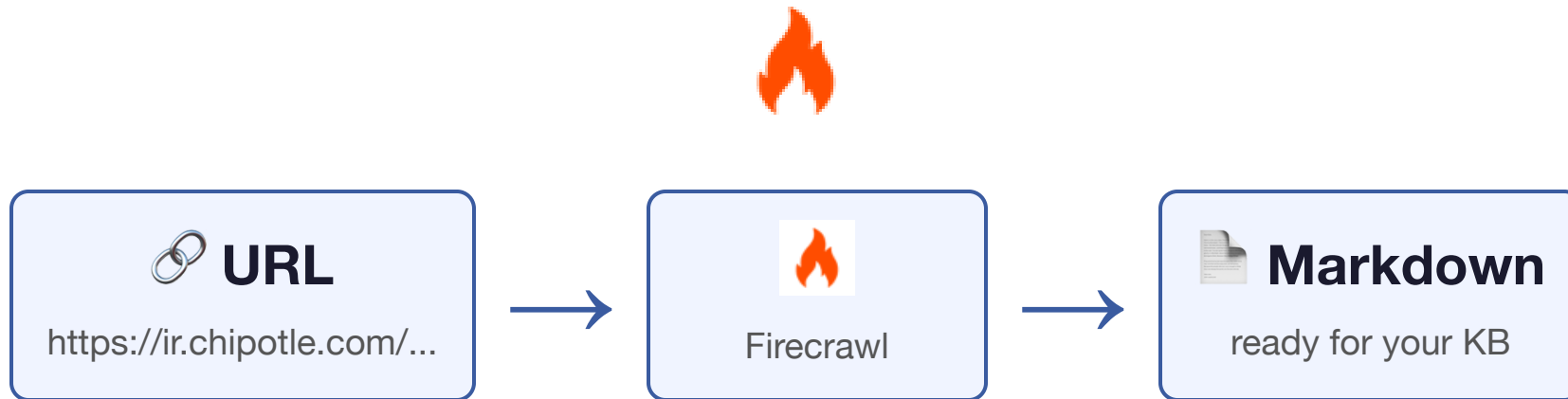
Tavily vs. Claude Code's WebSearch



Tavily (API)	WebSearch (chat)
Returns JSON	Returns a paragraph
For your pipeline	For you
Runs in GitHub Actions	Requires interactive Claude Code
Reproducible	Each answer is different

If a cron job needs the answer, use the API. If a human needs the answer, use the chat tool.

Firecrawl — URL → Clean Markdown



No BeautifulSoup. No `select` or `find`. No DOM inspection.

Last Year We Taught BeautifulSoup

- Parse HTML tag-by-tag
- Hunt for the right CSS selector
- Break every time the site redesigns



Last Year We Taught BeautifulSoup

- Parse HTML tag-by-tag
- Hunt for the right CSS selector
- Break every time the site redesigns

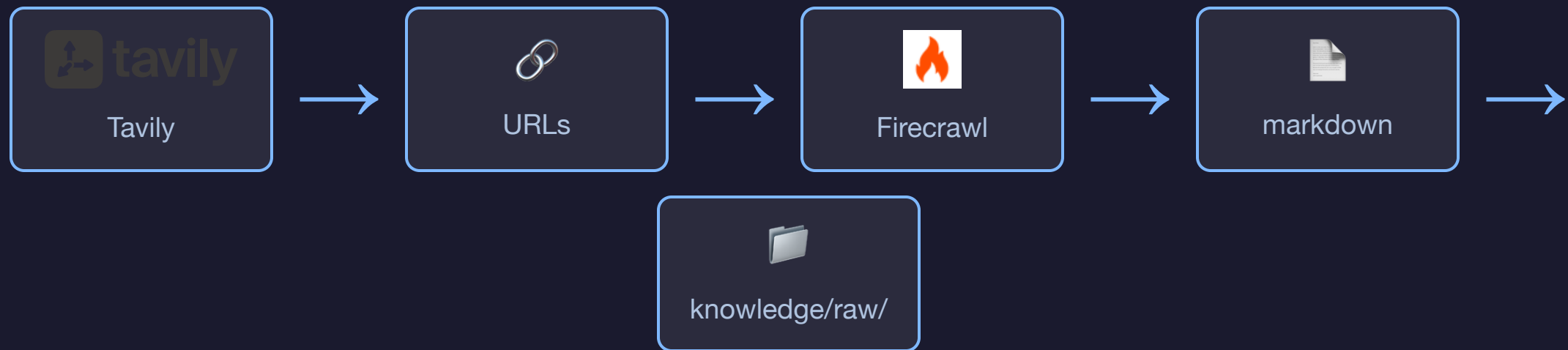
This year we don't. Firecrawl does all three in one call, and handles JavaScript rendering too. If you want BeautifulSoup for interview prep, read the docs — you do not need it for this project.



Build vs. Buy — Scraping Edition

Problem	Don't	Do
 Find relevant URLs	Write a crawler	Tavily
 Turn URL into text	Write a parser	Firecrawl
 Click through a login	Write a bot	Playwright

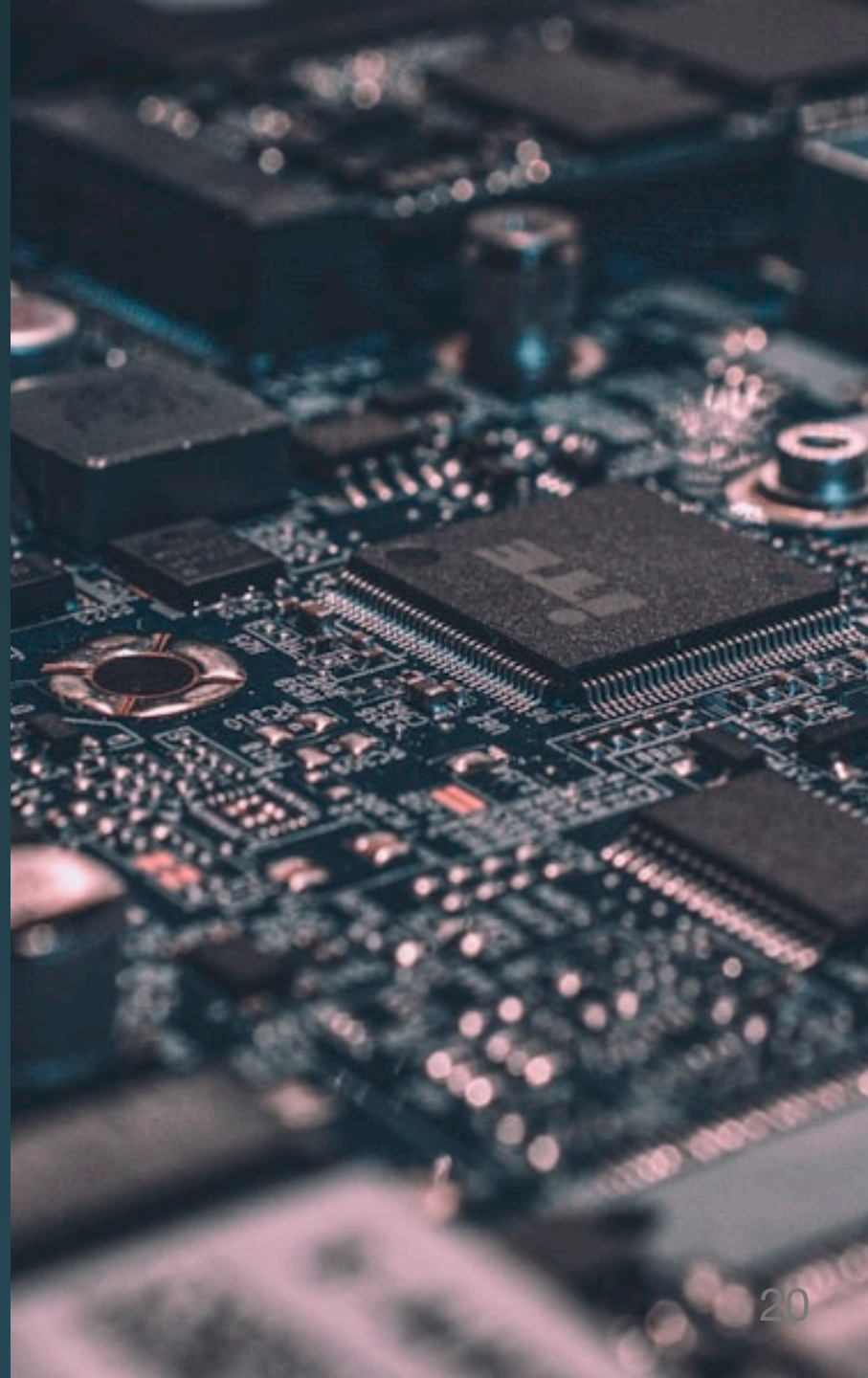
The Pipeline



One tool finds the pages. Another tool extracts the content. Your script glues them together.

Now the Upgrade: MCP

What if Claude Code could call Tavily and Firecrawl **directly**, without you writing Python?



Now the Upgrade: MCP



MCP servers (Model Context Protocol) extend Claude Code with new tools. Install one, and Claude Code can use it inside any prompt.

Python vs. MCP

Python (Part 02)

- ~35 lines of SDK calls
- Loops, file writes, sleeps
- Version-controlled
- Runs in GitHub Actions

MCP (Part 03)

"Find 5 URLs about Chipotle's recent earnings and save each as markdown in
knowledge/raw/."

Same result. No code.

Today's Demo Domain: Chipotle IR



IR = Investor Relations —
shareholder-facing content



Press releases, bios, earnings, filings

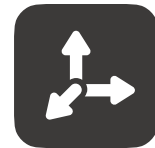


Legally required to be accessible



Great fit for a knowledge base

Accounts You Need



tavily

Service	URL	Free Tier
Firecrawl	firecrawl.dev	500 scrapes/month
Tavily	tavily.com	1,000 searches/month

Both: GitHub sign-in, no card. Under 2 minutes each.

Your Project Needs This



Milestone 02: ≥ 15
sources from 3+
sites



Pattern works for
any URL in a
browser



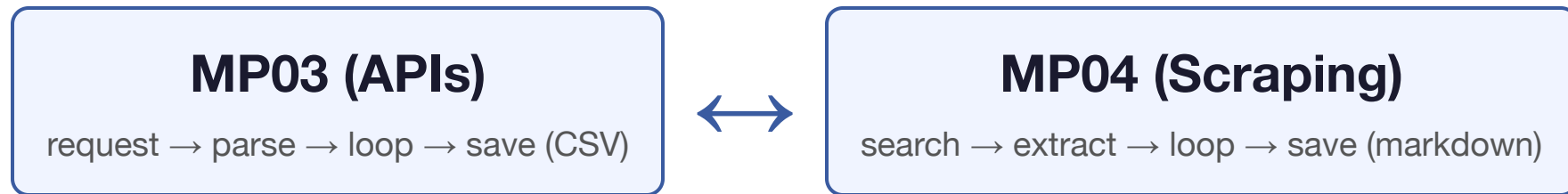
Your job posting
says what
content matters



Feeds your
knowledge base
wiki



The Pattern Transfers



Different tools. Same shape. If you learned MP03, you already know MP04.

Next up: code it together.